

Design Patterns

def: Решение на
тема възниква
проблеми; добри
практики за дизайн

→ 3 вида:

Creational

- object creation mechanisms

Structural

- assembling objects and classes into larger structures

Behavioral

- algorithms and responsibilities between objects

Singleton

Factory Method

Abstract Factory

Prototype

Composite

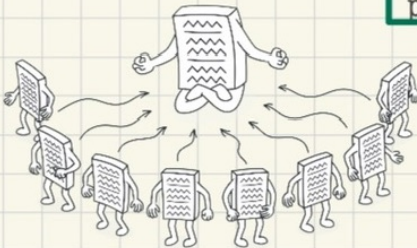
Flyweight

Command

Iterator

I. Singleton

Singleton is a creational design pattern that lets you ensure that a class has only one instance, while providing a global access point to this instance.

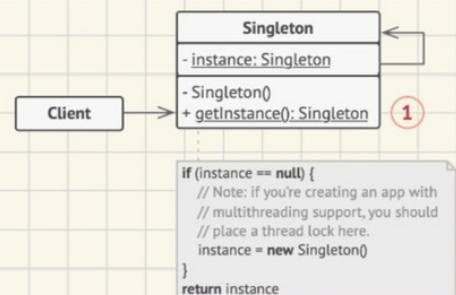


→ move and copy are deleted

- Make the default constructor private (prevent other objects from using the new operator)
- Create a static creation method that acts as a constructor. (calls the private constructor to create an object and saves it in a static field.)

Пример: String Pool

```
-struct DBProvider
-{
-    static DBProvider& getInstance()
-    {
-        static DBProvider obj(1234);
-        return obj;
-    }
-private:
-    DBProvider(int port) {}
-    DBProvider(const DBProvider&) = delete;
-    DBProvider& operator=(const DBProvider&) = delete;
-};
```



```

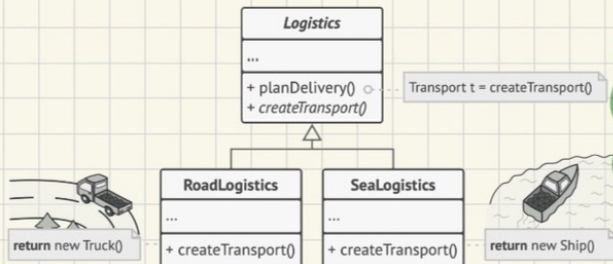
~struct DBProvider
~{
~    static DBProvider& getInstance()
~    {
~        static DBProvider obj(1234);
~        return obj;
~    }
~private:
~    DBProvider(int port) {}
~
~    DBProvider(const DBProvider&) = delete;
~    DBProvider& operator=(const DBProvider&) = delete;
~};

```

Made with Goodnotes

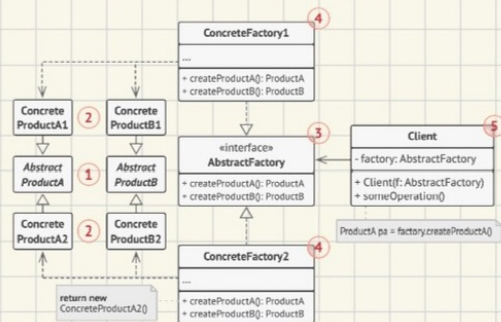
II. Factory

Factory Method is a creational design pattern that provides an interface for creating objects in a superclass, but allows subclasses to alter the type of objects that will be created.



Use the Factory Method when you don't know beforehand the exact types and dependencies of the objects your code should work with.

II. Abstract Factory



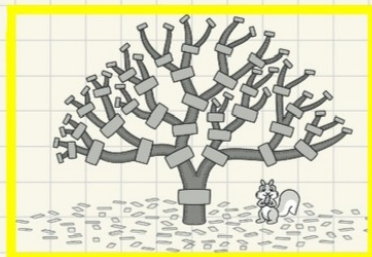
II. Flyweight

- зберігає більше об'єктів в пам'яті через споживання на об'єкти, які мають однакові частини

String Pool / Array Pool

II. Composite

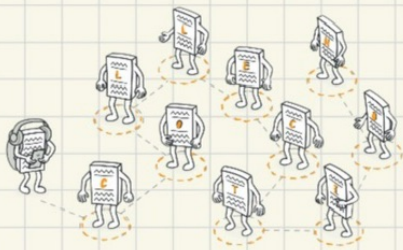
- дозволяє будувати складні структури, в яких кожен вузол може бути як листом, так і гілкою



Composite is a structural design pattern that lets you compose objects into tree structures and then work with these structures as if they were individual objects

Boolean Expression

I. Iterator



Iterator is a behavioral design pattern that lets you traverse elements of a collection without exposing its underlying representation (list, stack, tree, etc.).

```
306 -template <class IterType, class ElType>
307 -bool linearSearch(const IterType& begin, const IterType
308 -{
309 -    IterType current = begin;
310 -
311 -    while (current != end)
312 -    {
313 -        if (*current == el)
314 -            return true;
315 -        current++;
316 -    }
317 -    return false;
```

Functions needed:

- op++
- op--
- ==, !=
- (*iter) - връща el-m

→ Every collection has the following:

- begin() - returns iterator to the beginning
- end() - returns iterator to the end

II. Command

Command is a behavioral design pattern that turns a request into a stand-alone object that contains all information about the request. This transformation lets you pass requests as a method arguments, delay or queue a request's execution, and support undoable operations.

Type Casting

Ако качеството не е утвърдено → bad_cast

1. static_cast

! Не прави type checking и го ползваме само когато сме сигурни за типа.

Ако не става няма да се комп.

2. dynamic_cast

- Не сме сигурни за типа
- има runtime check
- по-бавно от static_cast

III. Const_cast

- за премахване на константността на укал или референция

За да стане това трябва подаденият обект да не е декларирани като константен при създаването.

Работи чрез виртуална таблица на обекта, който качваме и следва свързани с тази на обекта, към който качваме.

За да работи трябва иерархията да има поне 1 виртуална ф-я

IV. Reinterpret_cast

- от всеки укал към всеки укал
- няма проверки

Design patterns

Three types:

- **Structural** - за създаване на по-сложни обекти
- **Creational** - за създаване на обектите; отговор на въпроса за създаване
- **Behavioral** - комуникация м/у обектите

I. Creational

• Singleton

ОСИГУРЯВА САМО 1 ИНСТАНЦИЯ, КЪМ КОЯТО ИМА ДОСТЪП

String Pool

move и copy са изтривач

- + ИМА САМО 1 ИНСТАНЦИЯ НА КЛАСА
- + ИМА ГЛОБАЛЕН ДОСТЪП ДО ИНСТАНЦИЯТА
- + ОБЕКТЪТ СЕ ИНИЦИАЛИЗИРА ПРИ ДОСТЪПВАНЕ НА ОБЕКТА
- МНОГОНИШКОВО ПРОГРАМИРАНЕ
- МН. ЧЕСТО СЕ ОПРЕДЕЛЯ КАТО АНТИПАТЕРН (НЕ МОЖЕ ДА СЕ ПРЕНЕСЕ ПРИ ИЗПОЛЗВАНЕ С ДРУГ КОД, ОБВЪРЗАНО С КОНКРЕТНА ИНСТАНЦИЯ)

```
1 class SingletonClass {
2 private:
3     SingletonClass() {
4         std::cout << "SingletonClass was created" << std::endl;
5     }
6     ~SingletonClass() {
7         std::cout << "SingletonClass was destroyed" << std::endl;
8     }
9 public:
10     static SingletonClass& getInstance() {
11         static SingletonClass instance;
12         return instance;
13     }
14     SingletonClass(const SingletonClass& other) = delete;
15     SingletonClass& operator=(const SingletonClass& other) = delete;
16     void doStuff() {
17         std::cout << this << std::endl;
18     }
19 };
20
21 int main() {
22     {
23         std::cout << "hello";
24         SingletonClass& s1 = SingletonClass::getInstance();
25     }
26     SingletonClass& s2 = SingletonClass::getInstance();
27     s2.doStuff();
28     SingletonClass& s3 = SingletonClass::getInstance();
29     s3.doStuff();
30     SingletonClass& s4 = SingletonClass::getInstance();
31 }
```

Made with Goodnotes

• Factory

- статична функция (може да е в клас), която на база аргумент връща инстанция на клас

• - Factory

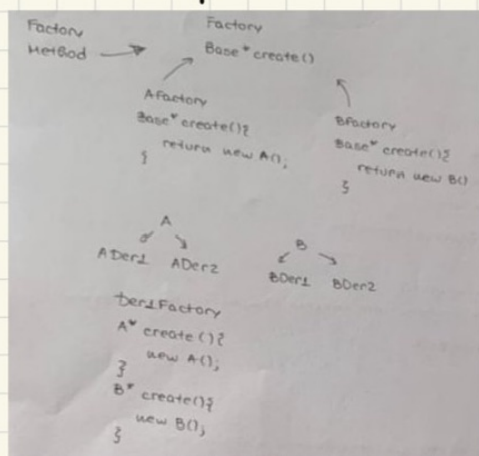
- статична функция (може да е в клас), която на база аргумент връща инстанция на клас

- Factory Method

- предоставя интерфейс с метод `create` метод. Всеки наследник предоставя имп-р на `create` като решава какъв обект да върне

- Abstract Factory

- предоставя интерфейс с `create` метод, за всеки обект, който създава. Мезу обекти са свързани логически



+ ГЕНЕРАЛНО МЯСТО ЗА СЪЗДАВАНЕ НА ОБЕКТИ

+ ПО-ВСЕКО ВРЕМЕ МОГА ДА ДОБАВЕ НОВ НАСЛЕДНИК НА BaseFactory (ФМ, АФ)

- 1 - 1 на BASE

- ПО-СЛОЖЕН КОД ФМ, АФ

- ГРИНА ЗА ИМВОТА НА Factory ФМ, АФ



• Prototype

- създава копие на обекта без да се интер-оп какъв тип е
→ clone()

→ от получ. иерархия

II. Structural

- Composite - структуриране на обекти в гърбовула стр-ра
- Flyweight → String Pool, хеш, дърво
 - + създава повече обекти в паметта като споделя общите им ресурси
 - + подобрява бързодействието, ако създава об-кт е мейка от-е

III. Behavioral

- Iterator - начин за работа с колекция без да се интер-оп каква е тя
* итератор - ук-л към конк-мен обект

→ endl(), begin()

```
ИТЕРАТОР - итератор  
Iterator it  
(*it) - връща текущия елемент  
op++ - отиди на следващия  
op-- - отиди на предишния  
op*  
op!=  
op==  
  
template<class IterType>  
bool Search (Iter begin, Iter end)  
{  
    while (begin != end)  
        if (*begin == searched)  
            return true;  
    begin++;  
    return false;  
}
```

• Command

Command is a behavioral design pattern that turns a request into a stand-alone object that contains all information about the request. This transformation lets you pass requests as a method arguments, delay or queue a request's execution, and support undoable operations.

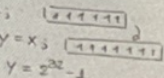
Type casting

→ преобразване от 1 тип в друг

int x = 7;
double y = x;



int x = -1;
unsigned y = x;



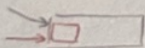
$y = 2^{32} - 1$

2) Base
↑
Der
Der* d = new D(),
Base* b = d; Base* b = (Base*)d
C-style cast

I Static_cast

Base* ptr = static_cast<Base*>(ptr) → вказател към Der
→ когато сме сигурни в типа, в който преобразване
(сигурни сме, че преобразва в правилен за вказателя тип)

⚠ Не прави runtime check
(Ако сбъркаме, то CRASH)

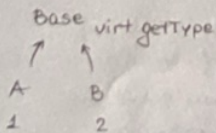


→ upcasting

→

if (ptr != nullptr)

if (Base* ptr) {
if (ptr->getType() == 1)
A* obj = static_cast<A*>(ptr);
→ downcasting
}



Base* ptr = new A();
B* b = static_cast<B*>(ptr);
⚡

Старо C++

```
if (void*, callback) {  
    callback(ptr, ...);  
}  
  
int x = 0;  
void* ptr = &x;  
if (ptr, [](void* ptr) {  
    int* data = static_cast<int*>(ptr);  
})
```

void* → може да е константа или не
→ не може да се преобразува
→ C-style cast

II dynamic_cast

- преобразване с runtime check
(ако работим с вказатели и преобразването е неуспешно, то dynamic_cast ще върне nullptr)
- не се използва за upcasting, защото тогава винаги ще е успешен и няма нужда от check
- използва се основно за downcast, когато не знаем какъв е типът



```
if (Base* ptr) {  
    if (A* aptr == dynamic_cast<A*>(ptr))  
        // A* aptr  
        // A* aptr == A  
    else if (B* bptr == dynamic_cast<B*>(ptr))  
        // B* bptr  
        // B* bptr == B
```

⚠ Проверката става през виртуална таблица

⚠ Down casting може да прави само ако имаме поне 1 виртуална таблица

```
if (Base& ref) {  
    A& ref = dynamic_cast<A&>(ref);  
    → Ако е неуспешен, хвърля std::bad_cast
```

III. const_cast

- манипулиране на константност на C++
с които работи през указатели и референци

⚠ Не го използваме!

- За да премахнем const, трябва да сме сигурни, че при създаването си този обект не е бил константен

```
int x = 10;
```

```
f(x)
```

```
f(const int* ptr)
```

```
int* n = const_cast<int*>(ptr);
```

```
(*n)++;
```

```
const int x = 10;
```

```
f(x);
```

```
f(const int* ptr)
```

```
int* n = const_cast<int*>(ptr);
```

U.B.

Константите се намират на друго място в паметта.

IV. reinterpret_cast

```
class X
```

```
int a;
```

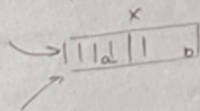
```
int b;
```

```
class Y
```

```
char ch[5];
```

```
X* ptrX = new X();
```

```
Y* ptrY = reinterpret_cast<Y*>(ptrX);
```



- реинтерпретация на памет

```
int* x;
```

```
file.write(reinterpret_cast<char const char*>(x));
```

V. C style cast

```
A* ptr = (B*)ptr2;
```

```
const_cast
```

```
static_cast
```

```
static_cast + const_cast
```

```
reinterpret_cast
```

```
▼ reinterpret_cast + const_cast
```